

Wideband Beamforming

Every line of the notebook explained — math first, then the code that does it

Contents

1	Why You Are Replacing <code>dsp_worker.py</code>	2
2	The Physics: How Sound Arrives Late	2
2.1	Geometry of a microphone array	2
2.2	The angular convention in the notebook	2
3	The Key Insight: A Delay Is a Phase Rotation	3
3.1	A pure tone in the time and frequency domain	3
3.2	What happens when you delay the signal	3
3.3	Corollary: phase of $X(f)$ is timing information	3
4	The Steering Matrix	4
4.1	Building it up from what we know	4
4.2	The notebook's code for this	4
4.3	What <code>[:, None, None]</code> does — broadcasting	5
5	The Beamforming Step: <code>einsum</code>	5
5.1	What we want to compute	5
5.2	What <code>np.einsum</code> does	6
5.3	The PHAT whitening step	6
5.4	The wideband sum	6
6	The Phase-Delay Demo (Notebook Cell 2)	7
7	The Frequency Bin Index Lookup	7
8	Old Code vs New Code, Side by Side	8
8.1	The old inner loop (your current code)	8
8.2	The new approach (steering matrix + <code>einsum</code>)	8
9	One Last Thing: <code>W.conj()</code> — Why Conjugate?	9

1 Why You Are Replacing `dsp_worker.py`

Your current `_srp_phat()` function has a loop that runs 120 times (once per mic pair), and inside that loop there is **another** Python loop over all 181 angles. This is slow because Python executes each iteration one at a time.

The notebook replaces both loops with a single matrix multiply. Before we can understand the matrix, we need to understand what it contains, which means we need to understand how a time delay becomes a complex number. That is the entire first half of this document.

Reading plan. Sections 2–4 are pure math — read them slowly and do not skip them. Section 5 is where the notebook’s actual code is explained line by line. Section 6 shows the exact changes to your `dsp_worker.py`.

2 The Physics: How Sound Arrives Late

2.1 Geometry of a microphone array

You have 16 microphones on a line, spaced $d = 0.05$ m apart. A sound source is far away at angle θ measured from the broadside (perpendicular to the array).

When sound arrives from angle θ , the wavefront hits each microphone at a slightly different time. Microphone n (counting from the centre) is at position $x_n = n \cdot d$ metres from the centre. The extra distance the wave must travel to reach mic n compared to the centre is:

$$\Delta x_n = n \cdot d \cdot \sin(\theta)$$

The time it takes sound to travel that extra distance is the **Time Difference of Arrival (TDOA)**:

$$\tau_n(\theta) = \frac{n \cdot d \cdot \sin(\theta)}{c}$$

where $c = 343$ m/s. So mic n hears the sound τ_n seconds **after** the centre mic (if the sound comes from the positive side).

Concrete example. $d = 0.05$ m, $\theta = 30^\circ$, $n = 7$ (the rightmost mic): $\tau_7 = 7 \times 0.05 \times \frac{\sin(30^\circ)}{343} = 7 \times 0.05 \times \frac{0.5}{343} \approx 0.00051$ seconds ≈ 24 samples at 48 kHz.

In your old code, this 24.5 gets rounded to 24 — you lose the 0.5 sample. In the new code, you keep it exactly.

2.2 The angular convention in the notebook

The notebook uses $\theta \in [0^\circ, 180^\circ]$ with $\cos(\theta)$ instead of $\sin(\theta)$. This is the **axial convention**: $\theta = 90^\circ$ means broadside (straight ahead), $\theta = 0^\circ$ means end-fire left, $\theta = 180^\circ$ means end-fire right.

Your `config.py` uses $\theta \in [-90^\circ, +90^\circ]$ with $\sin(\theta)$: $\theta = 0^\circ$ is broadside. These describe the same physical directions — just different coordinate choices.

$$\underbrace{\sin(\theta_{\text{your}})}_{\theta \in [-90, 90]} = \underbrace{\cos(\theta_{\text{notebook}})}_{\theta_{\text{notebook}} \in [0, 180]}$$

The dsp_worker.py replacement at the end of this document uses the sin convention to stay consistent with your existing SCAN_ANGLES and display code.

3 The Key Insight: A Delay Is a Phase Rotation

This is the single idea that makes the whole notebook work. Read this section until it is obvious.

3.1 A pure tone in the time and frequency domain

Consider a single tone at frequency f :

$$s(t) = \cos(2\pi ft)$$

Using Euler's formula, we can write this as the real part of a complex exponential:

$$s(t) = \text{Re}[e^{j2\pi ft}]$$

The Fourier transform of $s(t)$ at frequency f gives us a complex number $S(f)$. Its magnitude tells us **how loud** that frequency is. Its **angle** (phase) tells us **where in the cycle** we are at time $t = 0$.

3.2 What happens when you delay the signal

Now delay the same tone by τ seconds — this is what the microphone array does physically:

$$s(t - \tau) = \cos(2\pi f(t - \tau)) = \text{Re}[e^{j2\pi f(t - \tau)}]$$

Expanding:

$$e^{j2\pi f(t - \tau)} = e^{j2\pi ft} \cdot e^{-j2\pi f\tau}$$

The second factor $e^{-j2\pi f\tau}$ does not depend on time. In the Fourier domain this means:

$$S_{\text{delayed}}(f) = S(f) \cdot e^{-j2\pi f\tau}$$

A time delay τ in the time domain is exactly multiplication by $e^{-j2\pi f\tau}$ in the frequency domain. This is not an approximation. It is exact for any signal and any delay, including non-integer sample delays.

3.3 Corollary: phase of $X(f)$ is timing information

Claim: The phase angle $\varphi = \angle X(f)$ of the Fourier transform of a signal is directly proportional to the time delay at which that frequency component arrived.

Proof:

Let a signal $s(t) = e^{j2\pi f_0 t}$ be delayed by τ seconds: $s(t - \tau)$.

Its Fourier transform at f_0 is:

$$X(f_0) = \int_{-\infty}^{\infty} e^{j2\pi f_0(t - \tau)} e^{-j2\pi f_0 t} dt = e^{-j2\pi f_0 \tau}$$

The phase of this is:


```
# THE STEERING MATRIX
# shape: (100 freqs, 15 mics, 100 angles)
W = np.exp(
    -1j * d * k[:, None, None]          # (100, 1, 1)
    * mic[None, :, None]              # (1, 15, 1)
    * np.cos(steering_angles[None, None, :]) # (1, 1, 100)
)
```

4.3 What `[:, None, None]` does — broadcasting

NumPy broadcasting lets you multiply arrays of different shapes as long as their dimensions are compatible. The trick is to add size-1 dimensions with `None` (same as `np.newaxis`) so NumPy knows which axis to align.

Without broadcasting you would need three nested loops:

```
# Slow version with loops — equivalent to the one line above
W = np.zeros((100, 15, 100), dtype=complex)
for f_idx in range(100):      # loop over frequencies
    for n in range(15):        # loop over mics
        for a_idx in range(100): # loop over angles
            W[f_idx, n, a_idx] = np.exp(
                -1j * d * k[f_idx] * mic[n] * np.cos(steering_angles[a_idx])
            )
```

Broadcasting does all $100 \times 15 \times 100 = 150,000$ multiplications in one C call. The shapes broadcast like this:

<code>k[:, None, None]</code>	$\rightarrow (100, 1, 1)$
<code>mic[None, :, None]</code>	$\rightarrow (1, 15, 1)$
<code>cos(angles)[None, None, :]</code>	$\rightarrow (1, 1, 100)$
product	$\rightarrow (100, 15, 100) \leftarrow W \text{ shape}$

5 The Beamforming Step: `einsum`

5.1 What we want to compute

We have the steering matrix $\mathbf{W}$ of shape (F freqs, M mics, A angles). We have the measured signal $\mathbf{X}$ of shape (F freqs, M mics) — the FFT of each microphone’s recording at each frequency.

For each candidate angle θ , we want to compute the beamformed output: apply the conjugate of the steering weights to the signal and sum over all mics.

$$Y(f, \theta) = \sum_{n=0}^{M-1} W^*(f, n, \theta) \cdot X(f, n)$$

In matrix terms: for each frequency f , multiply the complex-conjugate of the f -th slice of W by the corresponding signal, and sum over the mic axis.

5.2 What `np.einsum` does

`np.einsum` is Einstein summation notation. You write a string that describes which axes to multiply and which to sum over, and NumPy handles the rest.

```
Y = np.einsum('fms, fm -> fs', W.conj(), signal_aligned)
```

Read the string 'fms, fm -> fs' as:

- First array has axes (f=freq, m=mic, s=steering angle) — this is `W.conj()`
- Second array has axes (f=freq, m=mic) — this is `signal_aligned`
- Output has axes (f=freq, s=steering angle) — this is `Y`
- The `m` axis appears in both inputs but **not** in the output — so NumPy sums over it

Expanded:

```
Y[f, s] = sum over m: W.conj()[f, m, s] * signal[f, m]
```

This is exactly the formula above. One line instead of three nested loops.

5.3 The PHAT whitening step

Before passing the signal into `einsum`, the notebook normalises each frequency bin to magnitude 1. This is the “PHase Transform” part:

```
# After FFT, before einsum:
Xf = np.fft.rfft(data, axis=1) # shape (N_CHANNELS, N_FFT//2+1)
Xf_phat = Xf / (np.abs(Xf) + 1e-10) # divide by magnitude
```

`np.abs(Xf)` computes $|X(f)|$ — the magnitude (loudness) at each frequency. Dividing removes the magnitude, leaving only the phase. After this, every frequency bin has magnitude 1 and its phase intact.

Why? Without PHAT, strong frequencies dominate. A 4 kHz tone with amplitude 10 contributes $10\times$ more to the beam power than a 1 kHz tone with amplitude 1. Their delays are equally informative for DOA estimation, so we do not want amplitude to matter. PHAT makes every frequency equally important for the timing (phase) information it carries.

5.4 The wideband sum

```
P = np.mean(np.abs(Y)**2, axis=0) # shape: (n_angles,)
```

`np.abs(Y)**2` computes the power at each (frequency, angle). `np.mean(..., axis=0)` averages across the frequency axis, giving total power at each angle across the whole band. The angle with the highest power is the DOA estimate.

Summary of the full pipeline:

1. FFT each channel $\rightarrow X(f, n)$
2. PHAT-whiten $\rightarrow \hat{X}(f, n) = \frac{X(f, n)}{|X(f, n)|}$
3. Extract the frequency bins matching your steering frequencies

4. $\text{einsum} \rightarrow Y(f, \theta) = \sum_n W^*(f, n, \theta) \cdot \hat{X}(f, n)$
5. Average power $\rightarrow P(\theta) = \text{mean}_f |Y(f, \theta)|^2$
6. $\hat{\theta} = \arg \max_{\theta} P(\theta)$

6 The Phase-Delay Demo (Notebook Cell 2)

This is the very first demo in the notebook. It proves the phase-equals-delay idea on a real signal before any array processing.

```
N = 4800
T = 1 / 48000
t = np.arange(0.0, N*T, T)
S = np.sin(2*np.pi*4000*t) + np.sin(2*np.pi*3000*t)

tf = fftfreq(N, T) # frequency axis in Hz
tf = fftshift(tf) # shift so 0 Hz is in the centre
Sf = fft(S) # complex spectrum
Sf = fftshift(Sf) # shift to match tf

# Apply a delay of 0.0001 seconds (= 4.8 samples at 48 kHz)
# Multiply each frequency bin by e^(-j 2pi f tau)
Sf = Sf * np.exp(-1j * 2 * np.pi * tf * 0.0001)

# Convert back to time domain
S_sh = ifft(fftshift(Sf))
```

What each function does:

- `fft(S)` — converts the N-sample time signal into N complex numbers, one per frequency. Same as `np.fft.fft`. Uses an $O(N \log N)$ algorithm.
- `fftfreq(N, T)` — builds the frequency axis: the k -th output of `fft` corresponds to frequency `fftfreq[k]` Hz.
- `fftshift(x)` — rearranges an FFT output so that zero frequency is in the middle instead of at index 0. Makes plots look natural (negative freqs on left, positive on right).
- `np.exp(-1j * 2 * np.pi * tf * 0.0001)` — this is the delay applied to every frequency at once. The shape of `tf` and `Sf` are both $(N,)$, so this is element-wise: each frequency gets its own phase rotation.
- `ifftshift` undoes the shift before calling `ifft`, which converts back to time.

The plot shows `S` and `S_sh` overlaid — they look identical except `S_sh` is shifted right by 0.0001 seconds. This confirms: multiplying by $e^{\{-j2\pi f \tau\}}$ in the frequency domain = shifting by τ in the time domain.

7 The Frequency Bin Index Lookup

This is the part that confused many people reading the notebook. In cell 4 there is this code:

```
tf = fftshift(fftfreq(N, T)) # the frequency axis of the full FFT

# k holds 100 wavenumbers from 1000 to 4001 Hz (as radians/metre)
```

```
# We need to find which FFT bin corresponds to each of those frequencies.
# The wavenumber  $k = 2\pi f/c$ , so the frequency in Hz is  $f = k*c/(2\pi)$ 

idx_list = [
    np.argmin(np.abs(tf - (ki * c / (2 * np.pi))))
    for ki in k
]
signal_aligned = signal.T[idx_list, :] # shape: (n_freqs, N_CHANNELS)
```

Why is this needed? The FFT gives you N bins — one for every integer multiple of the frequency resolution $\Delta f = \frac{1}{N \cdot T}$. Your steering matrix $\mathbf{W}$ was built for 100 specific frequencies between 1000 and 4001 Hz. Those 100 frequencies do not land exactly on FFT bin centres. `np.argmin(np.abs(tf - target))` finds the nearest bin to each target frequency.

`signal.T` transposes from $(N_CHANNELS, N_FFT)$ to $(N_FFT, N_CHANNELS)$ so that indexing `[idx_list, :]` picks out the right frequency rows, giving shape (100 freqs, 16 mics) — matching what `einsum` expects.

This is where your old code and the notebook diverge most. Your old code used `np.fft.rfft` which only gives you positive frequencies (N

8 Old Code vs New Code, Side by Side

8.1 The old inner loop (your current code)

```
for i in range(N_CHANNELS): # 16 iterations
    for j in range(i + 1, N_CHANNELS): # up to 15 iterations = 120 total
        Xi = np.fft.rfft(data[i])
        Xj = np.fft.rfft(data[j])
        GCC = Xi * np.conj(Xj)
        PHAT = GCC / (np.abs(GCC) + 1e-10)
        gcc_t = np.real(np.fft.irfft(PHAT)) # back to time domain

    for a_idx, tau in enumerate(delays): # 181 iterations
        P[a_idx] += gcc_t[int(tau % n_fft)] # integer index lookup
```

What is slow: 120 Python loop iterations, each calling `rfft` + `irfft`. 181 inner Python iterations per pair for the index lookup. Total: 21,720 Python-level operations per frame.

What loses accuracy: `int(tau) % n_fft` — the delay is rounded to the nearest sample. At 48 kHz and 5 cm spacing, each sample represents about 7 mm of spatial resolution. Small angles near 0° have delays well under 1 sample and get rounded to 0.

8.2 The new approach (steering matrix + `einsum`)

```
# Done ONCE at startup — precompute  $\mathbf{W}$  of shape  $(N\_FREQS, N\_CHANNELS, N\_ANGLES)$ 
W = np.exp(-1j * MIC_SPACING * k[:, None, None]
           * mic[None, :, None])
```



```

* np.sin(angles)[None, None, :])

# Per frame — the entire DSP is these 5 lines:
Xf    = np.fft.rfft(data, axis=1)      # one call, all channels
Xf_phat = Xf / (np.abs(Xf) + 1e-10)    # PHAT whitening
sig    = Xf_phat[:, self._freq_idx].T  # select steering freqs
Y      = np.einsum('fma, fm->fa', self._W.conj(), sig) # beamform
P      = np.mean(np.abs(Y)**2, axis=0)  # wideband power

```

What is fast: One rfft call for all channels. One einsum (internally a BLAS matrix multiply). Two element-wise operations. Zero Python loops.

What is accurate: The exponent $e^{-(j k d n \sin(\theta))}$ is exact for any real-valued θ . There is no rounding.

9 One Last Thing: $W.conj()$ — Why Conjugate?

The steering vector $W_{f,n,\theta} = e^{\{-j k d n \sin(\theta)\}}$ encodes the delay that sound **from** angle θ **experiences** at mic n .

When we want to **steer the beam toward** θ — i.e., ask “does the signal match what we would expect if the source were at θ ?” — we need to **undo** those delays. Undoing a phase rotation of $-\varphi$ means applying $+\varphi$, which is the complex conjugate.

$$W^*(f, n, \theta) = e^{\{+j k d n \sin(\theta)\}}$$

Multiplying the received signal (which was delayed by $e^{\{-j k d n \sin(\theta)\}}$) by the conjugate steering weight cancels the delay:

$$e^{\{-j k d n \sin(\theta)\}} \cdot e^{\{+j k d n \sin(\theta)\}} = e^0 = 1$$

When the signals from all mics add up coherently (phases cancelled), you get a large $|Y|^2$. For all other angles, the phases do not cancel and they partially destructively interfere, giving smaller power.

The whole algorithm in one sentence: Build a matrix of complex numbers that represent the expected delays for every mic at every angle, take its conjugate (to undo those delays), multiply by the received signal, and the angle where everything lines up is your source direction.